

Architectural Design & Implementation Document

1. Specification Analysis & Requirements Breakdown

(Official Project Specification)

This specification is related to a Linux Kernel Module (LKM) implementing a system call throttling mechanism. The LKM should offer a device driver for supporting the registration/deregistration of:

- program names (executable names)
- user IDs
- syscall numbers (according to x86-64 specification)

Such registration can be supported via the `ioctl(...)` system call. The registered syscall numbers are used to indicate that the corresponding system calls can be critical for various aspects, like scalability, performance or security.

Each time one of the registered syscall is invoked by a program that is registered by the device driver or by a user (effective user-ID) that is registered by the device driver, the real execution of the system call needs to be controlled by a **monitor** offered by the LKM.

In particular, the LKM monitor can be configured to indicate what is the maximum number MAX of registered syscalls that can be actually invoked in a wall-clock-time window of 1 second by the registered program-names/user-IDs. If the actual volume of invocations per wall-clock-time unit exceeds MAX, then the invoking threads need to be temporarily blocked, thus preventing their actual syscall invocation.

The syscalls managed by the LKM can be of whatever nature. Hence they can be either blocking or non-blocking.

The device driver also needs to offer the possibility to verify at each time instant what are the registered program names, user-IDs and syscall numbers. Also, the update of such information can be only carried out by a thread which is running with effective user-ID set to the value 0 (root).

Also, the device driver needs to support the possibility to turn the syscall monitor off/on along time. If it is turned off, no limit on the frequency of registered syscalls invocations per wall-clock-time unit gets applied. Additionally, on the basis of the selected value for MAX, the device driver should also provide data related to:

- The peak delay for the actual execution of an invoked system call, and the corresponding program-name and user-ID
- The average and peak numbers of threads that had to be blocked

It is expected that the user level code for configuring the LKM monitor, and for testing the correct behavior of the developed LKM software is delivered as part of the project.

1.1 Triggering Logic Analysis

The specification defines a precise composite condition for the throttling mechanism. The monitor must act as an active filter between the user-space request and the kernel execution.

The activation logic is formalized as follows:

$$Throttle_{\Leftarrow} (SyscallID \in S) \wedge ((UserEUID \in U) \vee (ProgramName \in N))$$

Where S, U, N are the sets of registered syscalls, effective User-IDs, and program names respectively.

Implication: The system must strictly distinguish between a registered syscall invoked by a generic user (Allowed) and a registered syscall invoked by a monitored user (Throttled). PIDs are excluded as they are not explicitly required by the specification ("program names" != "process IDs"). Filtering non-matching entities ensures that core OS components (e.g., desktop environments relying heavily on getpid) are not inadvertently starved.

1.2 The “Wall-Clock” Constraint

The throughput limit is defined within a "1-second wall-clock window".

- **Timing Requirement:** Since the enforcement window has a granularity of one second, acquiring high-resolution timestamps for every monitored syscall invocation would introduce unnecessary overhead in the kernel hot path.
- **Implementation:** The module uses the kernel timekeeping subsystem and represents the enforcement window through jiffies. The variable `window_start_jiffies` stores the beginning of the current window, while `HZ` represents approximately one second in jiffies. High-resolution timestamps obtained through `ktime_get()` are used only for measuring the delay of invocations that are actually blocked.

1.3 Identity Management

- **Program Names:** The kernel stores the executable name in `current->comm` (limited to `TASK_COMM_LEN`, typically 16 chars). We use this for lookups. Although the specification informally refers to "executable names", in-kernel matching is implemented against `current->comm`, i.e. the process command name maintained by the kernel task structure.
- **User IDs:** The specification mandates checking the **Effective User-ID (EUID)**. This is crucial as it reflects the current privilege set (e.g., during `setuid` execution), distinct from the Real UID.
- **Syscall Numbers:** On x86-64, the syscall number is initially supplied through the `RAX` register. Within the saved register state, `orig_ax` preserves the original syscall number, whereas `ax` is used for the syscall return value and may be modified during syscall-entry.

processing. Therefore, the wrapper reads `regs->orig_ax` to identify the intercepted syscall safely.

2. Architectural Decisions

2.1 Interception Mechanism: Ftrace with IPMODIFY

Direct syscall-table modification was initially considered. However, this approach requires locating a non-exported internal symbol, modifying write-protected kernel memory, and depending on implementation details that may change across kernel versions.

- **Decision:** The final implementation uses Ftrace with the `FTRACE_OPS_FL_IPMODIFY` flag.
- **Justification:** When a syscall is registered, the module obtains its original target address from the syscall table and installs an Ftrace filter on that address. The Ftrace callback modifies the saved instruction pointer, `regs->ip`, redirecting execution to the common throttling wrapper. The syscall table is only read to obtain original function addresses; its entries are never modified. This avoids direct patching of protected kernel memory and provides reliable interception on the target Linux 5.15 kernel configuration.

2.2 Data Structures: The Scalability Challenge

The rule lookup is executed in the syscall hot path. A linear $O(N)$ scan would make the lookup cost grow with the number of configured rules.

- **Decision:** The implementation uses the kernel hash-table API provided by `<linux/hashtable.h>`.
- **Hash Keys:** Syscall numbers and EUIDs use numeric keys, while program names use a string hash.
- **Complexity:** Lookup has an average-case complexity close to $O(1)$. Program-name collisions are resolved through a final string comparison.

2.3 Concurrency Control: RCU (Read-Copy-Update)

The module follows a read-mostly access pattern: syscall wrappers frequently inspect rules, whereas configuration updates through `IOCTL` are comparatively rare.

- **Readers (Syscalls):** The syscall wrapper uses `rcu_read_lock()` and RCU-aware hash-table

traversal to inspect rules without acquiring a blocking mutex.

- **Writers (Configuration):** Configuration operations are serialized through `conf_mutex`. Rules are logically removed from the hash table and their memory is reclaimed only after the relevant RCU grace period, preventing readers from accessing already-freed objects.

RCU therefore allows readers and configuration updates to overlap safely while keeping the syscall fast path free from blocking locks.

2.4 Throttling Logic & Reentrancy Protection

- **Global Limit:** Since MAX applies to the aggregate volume of matching syscall invocations system-wide, the implementation uses a global `atomic64_t` counter. This provides strict aggregate enforcement, although it may introduce more contention than an approximate per-CPU solution.
- **Race Condition Mitigation (Double Reset):** Multiple CPUs may detect the expiration of the one-second window simultaneously. The reset is therefore protected by `window_reset_lock`. After acquiring the spinlock, the expiration condition is checked again because another CPU may already have completed the reset.
- **Reentrancy Protection:** When the wrapper invokes the original syscall function, that function still passes through the Ftrace instrumentation point. To prevent infinite redirection, the callback inspects `parent_ip`. If the caller address belongs to the LKM, as determined by `within_module(parent_ip, THIS_MODULE)`, the callback leaves the instruction pointer unchanged. This allows the original syscall to execute in its normal scheduling context without disabling preemption, including when the syscall may block.

3. Implementation Details

3.1 Global vs Per-CPU State

Different synchronization strategies are used according to the required consistency guarantees:

Throttling Counter: The enforcement counter is a global `atomic64_t`, ensuring that all matching invocations consume the same system-wide quota.

- **Blocking Statistics:** Each CPU maintains a local `blocked_count` through `DEFINE_PER_CPU`. These counters record blocking events, namely entries into the wait path. They do not necessarily represent distinct threads because the same thread may be blocked again after a retry. The per-CPU values are aggregated when statistics are requested through `IOCTL`.
- **Peak Record:** The maximum measured delay, the associated EUID, the command name, and the peak number of blocking events in a window are stored in the shared `sc_peak_record` structure. A spinlock protects the consistency of these related fields.

3.2 Blocking Mechanism & Anti-UAF Protection

Threads exceeding the configured quota are suspended through a kernel wait queue rather than using busy waiting.

- **Blocking State:** The wrapper uses `wait_event_interruptible_timeout()`, placing the thread in `TASK_INTERRUPTIBLE`. The timeout corresponds to the remaining portion of the current enforcement window.
- **Wake-Up and Retry:** When the timeout expires, the wrapper repeats the complete throttling check because several waiting threads may resume and compete for the new quota simultaneously. The module also calls `wake_up_all()` when the window is reset and when monitoring is disabled. Disabling the monitor immediately satisfies the wait condition; during normal throttling, threads primarily resume through expiration of the calculated timeout.
- **Signal Handling:** Because the wait is interruptible, an incoming signal may cause it to return `-ERESTARTSYS`. The current implementation applies a fail-open policy in this case and allows the original syscall to execute.
- **Safe Unloading:** During module removal, monitoring is first disabled and waiting threads are awakened. Ftrace hooks are then unregistered to prevent new redirections into the wrapper. After an RCU grace period, hook pointers can be unpublished and their structures reclaimed safely. The module subsequently waits for `active_threads` to reach zero before removing the remaining rules and device resources. RCU protects the lifetime of shared objects, whereas `active_threads` tracks complete wrapper executions.

3.3 Identity Management (String Handling)

Comparing the current command name against every registered name would require a linear scan.

- **Optimization:** The module first computes the hash of `current->comm` and searches only the corresponding hash-table bucket. When a rule has the same type and hash key, `strcmp()` is used to compare the actual command name and resolve possible hash collisions. This provides average-case lookup complexity close to $O(1)$.

4. User-Space Interface (API)

Communication between user space and the kernel module is implemented through the `/dev/sc_throttler` character device and a set of IOCTL commands. The current implementation requires effective user-ID 0 for every IOCTL operation, which is stricter than the minimum requirement that explicitly restricts configuration updates to root.

IOCTL Command	Arguments	Description
---------------	-----------	-------------

IOCTL_ADD_RULE	struct sc_conf *	Registers a Syscall, EUID, or Program Name.
IOCTL_DEL_RULE	struct sc_conf *	Removes a specific registered rule.
IOCTL_SET_MAX	unsigned long	Sets the maximum aggregate number of matching invocations allowed in each one-second enforcement window.
IOCTL_SET_ONOFF	int	Enables or disables enforcement. Disabling the monitor also wakes waiting threads.
IOCTL_GET_STATS	struct sc_stats *	Returns aggregated blocking-event statistics, peak delay, and the EUID and program name associated with the peak delay.
IOCTL_LIST_RULES	char*	Serializes all active rules to user-space.

5. Correctness & Security Testing

5.1 Functional Verification

- **Rule Enforcement & Independence:** Demonstrated that unregistered users and processes are not throttled, while registered targets are throttled according to the configured MAX limit within the module's jiffies-based 1-second enforcement window.
- **Wall-Clock Precision:** Validation showed that non-throttled calls complete immediately, whereas calls exceeding MAX are delayed until the current jiffies-based 1-second window

expires. This provides an efficient wall-clock approximation suitable for the kernel hot path.

- **Access-Control Verification:** By default, a non-root process is denied access to `/dev/sc_throttler` by the device permissions. In addition, the IOCTL handler independently verifies that `current_euid()` is zero and returns `-EPERM` to an unauthorized caller that reaches the handler.

5.2 Concurrency & Stress Tests

- **Pthread Barrier Burst Simulation:** Fifty threads were synchronized through `pthread_barrier_wait()` so that they invoked the monitored syscall almost simultaneously. With `MAX = 5`, exactly five invocations completed immediately, while forty-five experienced throttling.

The statistics reported 225 cumulative blocking events. This value does not represent 225 distinct threads: after each window, only five waiting invocations obtain quota, while the remaining ones enter the wait path again. Therefore:

$$45 + 40 + 35 + 30 + 25 + 20 + 15 + 10 + 5 = 225$$

This test exercises the global atomic counter, the synchronized window reset, and the retry logic under concurrent contention.

-

5.3 Edge Cases

- **Dynamic Command-Name Change:** The test uses `prctl(PR_SET_NAME)` to modify the command name stored in `current->comm` at runtime. The module correctly matches and throttles the newly assigned name. This also demonstrates that `current->comm` is mutable and should not be considered an immutable executable identity.
- **Hot-Removal Survival:** The module was removed while a process was actively being throttled. The teardown procedure disabled monitoring, awakened waiting threads, disconnected the Ftrace hooks, waited for RCU readers and active wrapper executions to drain, and then reclaimed the remaining resources. The waiting process resumed and terminated normally, while no kernel warning, use-after-free report, or scheduling-while-atomic error was observed during the test.